

Department of Artificial Intelligence & Data Science

HALLUCINATION DETECTION IN LARGE LANGUAGE MODELS: LITERATURE SURVEY, RESEARCH GAP ANALYSIS AND A LIGHTWEIGHT DETECTION FRAMEWORK

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted By

GOWTHAM C – 814724243047

BALAJI S – 814724243022

DHARSHAN K S – 814724243035

GURUMOORTHI A – 814724243049

HERMAN JOEL RAJ J – 814724243059

Under the Guidance of

Mrs. R. Kalaiyarasi

SRM TRP ENGINEERING COLLEGE

Academic Year: 2026-27 (ODD)

SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that GOWTHAM C (814724243047), BALAJI S (814724243022), DHARSHAN K S (814724243035), GURUMOORTHI A (814724243049) and HERMAN JOEL RAJ J (814724243059) from **Third year AI&DS** has successfully completed the work titled **"Hallucination Detection in Large Language Models: Literature Survey, Research Gap Analysis and a Lightweight Detection Framework"** at SRM TRP Engineering College, Tiruchirappalli, in the partial fulfilment of the Bachelor of Technology in Artificial Intelligence & Data Science.

Mrs. R. Kalaiyarasi
(Faculty In-charge)
Artificial Intelligence & Data Science

Dr. R. Perumal Raja
(Head of Department)
Artificial Intelligence & Data Science

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that we take this opportunity to acknowledge the invaluable support and guidance we have received throughout the course of this project. This journey has been both intellectually enriching and personally rewarding, significantly enhancing our understanding of large language models, their failure modes, and the practical challenges of making them trustworthy.

We would like to express our heartfelt thanks to our faculty in-charge, Mrs. R. Kalaiyarasi, and to the Head of the Department, Dr. R. Perumal Raja, for their guidance, encouragement, and constructive feedback during the development of this project. Their insights were instrumental in shaping the direction and quality of the work. We also extend our appreciation to the Department of Artificial Intelligence & Data Science for fostering an environment that promotes learning, innovation, and academic growth. The resources and opportunities provided played a vital role in the successful completion of this project.

A special note of thanks goes to the supporting staff for their cooperation and timely assistance, which ensured that the process remained smooth and efficient at every stage. Finally, we express our deepest gratitude to our parents for their constant support, encouragement, and belief in our abilities. This project has not only strengthened our knowledge but has also emphasized the importance of dedication, perseverance, and continuous learning. We remain sincerely thankful to everyone who contributed, directly or indirectly, to the successful completion of this project.

Academic Year: 2026-2027

GOWTHAM C – 814724243047

BALAJI S – 814724243022

DHARSHAN K S – 814724243035

GURUMOORTHI A – 814724243049

HERMAN JOEL RAJ J – 814724243059

TABLE OF CONTENTS

Sr. No	Title	Page No.
1	Acknowledgement	3
2	Abstract	5
3	Introduction	6
4	System Requirements	8
5	Literature Survey	9
6	Research Gap Identification	12
7	Methodology	13
8	Implementation	16
9	Mapping to Sustainable Development Goal 9	18
10	Conclusion and Future Work	19
11	References	19

ABSTRACT

Large Language Models (LLMs) have become core components of modern digital infrastructure, powering customer-service chatbots, decision-support tools, data-processing pipelines and code-generation assistants. Despite their fluency, LLMs frequently produce statements that sound plausible but are factually wrong or unsupported by the provided context, a failure known as hallucination. Because hallucinated output can silently propagate into downstream systems, reliable hallucination detection has become a prerequisite for the safe industrial adoption of generative AI.

This project presents a literature survey and research gap analysis on hallucination detection in LLMs. The foundational work studied is "Detecting hallucinations in language models using semantic entropy" published in Nature (2024), which separates a model's linguistic uncertainty from its factual uncertainty by clustering sampled answers according to meaning, using bidirectional entailment checks, and computing entropy over the resulting meaning clusters. The method reaches an AUROC of 0.790 without task-specific training. The survey further compares nine additional recent studies covering taxonomies of hallucination, internal-representation probing, spectral analysis of attention maps, adaptive token selection, contrastive learning, faithfulness metrics and hybrid static/dynamic analysis of generated code.

The analysis identifies an efficiency gap: semantic entropy requires sampling ten or more responses per prompt and running additional Natural Language Inference classification, which is prohibitively expensive for time-critical applications, and it provides no guarantee when a model is confidently wrong and consistently repeats the same incorrect answer. Based on this gap, the project proposes a lightweight, single-pass hallucination detection architecture that fuses token-level confidence, internal-representation features and an adaptive token-selection mechanism, together with a guard intended to flag confidently wrong responses. The work is mapped to United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure), since trustworthy and computationally affordable AI is essential for resilient, innovation-driven digital infrastructure.

Keywords: Deep Learning, Hallucination Detection, Large Language Models, Semantic Entropy, Uncertainty Estimation, SDG 9.

INTRODUCTION

1.1 Background

Advances in deep learning, and in particular the transformer architecture, have produced Large Language Models capable of writing fluent text, answering questions, summarising documents, translating languages and generating program code. Organisations across industry are integrating these models into their digital infrastructure for data processing, decision making and customer interaction. The models owe their versatility to training on very large text corpora, which allows them to capture statistical regularities of language and a substantial amount of world knowledge.

However, an LLM is fundamentally a probabilistic next-token generator. It is optimised to produce text that is likely given its context, not text that is verified to be true. As a result, a model may generate statements that are grammatically perfect and delivered with complete confidence, yet are fabricated, outdated, or contradictory to the source material it was given. This behaviour is widely referred to as **hallucination**.

1.2 What is Hallucination in LLMs?

Hallucination in an LLM is the generation of content that is nonsensical, factually incorrect, or unfaithful to the input or to established knowledge. The surveys studied in this project organise hallucination into two broad families [2], [3]:

- **Factuality hallucination:** the output conflicts with real-world facts (factual contradiction) or contains claims that cannot be verified because they are invented (factual fabrication).
- **Faithfulness hallucination:** the output deviates from the user's instruction, from the supplied context, or from its own internal logic (instruction, context and logical inconsistency).

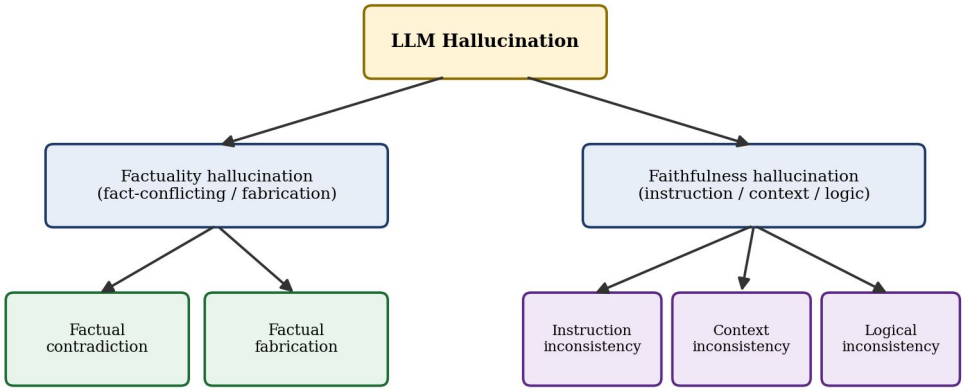


Figure 1.1: Taxonomy of LLM hallucination (adapted from the surveys in [2], [3])

1.3 Why Detecting Hallucination Matters

Hallucination is not merely an academic curiosity. A customer-service chatbot that invents a refund policy, a coding assistant that fabricates a non-existent library function, or an analytics pipeline that summarises data incorrectly can each cause financial loss, security risk or loss of user trust. When such outputs feed automated downstream applications, a single faulty response can cascade into infrastructure-level failures [2], [3]. Detection is therefore a prerequisite for safe deployment: if the system can recognise

when a response is unreliable, it can withhold the answer, ask for clarification, trigger retrieval-augmented generation, or escalate to a human reviewer.

1.4 Problem Statement

Existing hallucination detectors that rely on sampling multiple responses, such as semantic entropy, are accurate on many benchmarks but computationally expensive. They multiply inference cost by an order of magnitude, which makes them unsuitable for real-time use, and they fail in the important case where a model has confidently learned incorrect information. There is a need for a lightweight architecture that can detect hallucinations in a single forward pass, without sampling multiple responses, while also offering a principled way of catching confidently wrong answers.

1.5 Objectives

1. To study the foundational semantic entropy method for hallucination detection and understand its strengths and limitations.
2. To conduct a comparative literature survey of ten recent, high-quality studies on hallucination taxonomy, evaluation and detection.
3. To identify a clear research gap in the efficiency and reliability of current detection techniques.
4. To propose a lightweight, single-pass detection architecture that addresses the identified gap.
5. To map the contribution of the work to United Nations Sustainable Development Goal 9.

1.6 Scope of the Work

This report focuses on detection of hallucination in text generated by LLMs, with emphasis on question-answering style factual generation. Related areas such as mitigation of hallucination (for example retrieval-augmented generation and fine-tuning) and hallucination in code generation are discussed where they inform the survey, but the central contribution is the analysis of detection methods and the design of an efficient detection framework.

1.7 Organisation of the Report

Chapter 2 lists the hardware and software requirements. Chapter 3 presents the literature survey, including a comparative analysis of ten papers. Chapter 4 identifies the research gap. Chapter 5 describes the methodology, covering semantic entropy and the proposed lightweight framework. Chapter 6 describes the implementation plan, algorithms and illustrative outputs. Chapter 7 maps the work to SDG 9, and Chapter 8 concludes with future directions.

SYSTEM REQUIREMENTS

The tools below are recommended for implementing and evaluating the detection experiments described in this report, including running a baseline semantic-entropy pipeline and the proposed lightweight detector.

Category	Requirement	Description
Operating System	Windows / Linux / macOS	Any OS supporting Python and deep learning frameworks. Linux is preferred for GPU drivers and better performance.
Processor (CPU)	Intel i5/i7 or AMD Ryzen 5/7 (or higher)	Multi-core processors speed up tokenisation, data loading and evaluation scripts.
GPU (Recommended)	NVIDIA GPU with 12 GB+ VRAM (e.g., RTX 3060 or higher)	Needed to run LLM inference, extract hidden states and train the lightweight detection head with CUDA acceleration. Smaller open models can run on lower-memory GPUs with quantisation.
RAM	Minimum 16 GB (Recommended: 32 GB)	Required for holding datasets, cached model outputs and sampled responses during baseline comparison.
Storage	Minimum 50 GB free space	For model checkpoints, benchmark datasets, extracted feature files and experiment logs.
Programming Language	Python 3.10+	Standard language for deep learning research with extensive library support.
Deep Learning Framework	PyTorch	Used to run LLMs, access logits and hidden states, and train the fusion head.
NLP Libraries	Hugging Face Transformers, Datasets, sentence-transformers	Provide pre-trained LLMs, benchmark loaders and an entailment (NLI) model for the semantic-entropy baseline.
Data Handling Libraries	NumPy, Pandas, scikit-learn	Used for data manipulation, feature handling, and computing AUROC, precision and recall.
Visualization Tools	Matplotlib, Seaborn	For ROC curves, entropy distributions, and result plots.
Development Environment	Jupyter Notebook / Visual Studio Code	For writing, testing and debugging experiments.
Benchmarks / Datasets	Question-answering and hallucination benchmarks (e.g., TriviaQA, HaluEval)	Provide questions, reference answers and hallucination labels for training and evaluating detectors.

LITERATURE SURVEY

This chapter reviews ten recent studies on hallucination in LLMs. The foundational paper, on semantic entropy, is examined in depth; the remaining papers are grouped into surveys and taxonomies, white-box detection methods, efficient detection methods, and evaluation and domain-specific work.

3.1 Foundational Paper: Semantic Entropy [1]

Farquhar, Kenton, Mikulik, Marcus and Legg published "Detecting hallucinations in language models using semantic entropy" in Nature in 2024. The paper addresses one of the hardest problems in natural language processing: distinguishing a model's uncertainty about how to phrase an answer from its uncertainty about the facts.

Earlier uncertainty-based approaches compute entropy over the tokens or whole sequences a model generates. These approaches are sensitive to wording: a model that answers "Paris" in one sample and "It is Paris" in another appears to produce two different outcomes, even though both express the same fact. Token-level entropy therefore overestimates uncertainty when the model is actually confident but varies its phrasing.

Semantic entropy resolves this by measuring uncertainty over **meanings** rather than strings. Several answers are sampled for the same prompt, and an entailment model checks whether each pair of answers mutually entails the other (bidirectional entailment). Answers that entail each other are placed in the same meaning cluster. Entropy is then computed over the probability mass of each cluster. If the model consistently produces answers with the same meaning, semantic entropy is low and the model is confident; if the answers spread across several incompatible meanings, semantic entropy is high and the model is likely to be hallucinating.

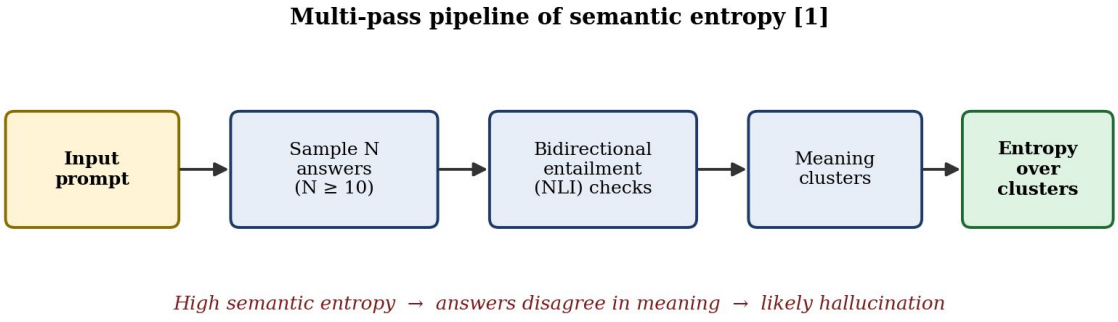


Figure 3.1: Multi-pass semantic entropy pipeline [1]

As reported in the comparative table of this study, the method achieves high-confidence detection with an AUROC of 0.790 without task-specific training, which makes it broadly applicable across domains. Its limitations, however, are the high computational overhead and its failure on confidently wrong data, both of which motivate the research gap in Chapter 4.

3.2 Surveys and Taxonomies of Hallucination [2], [3], [8]

Li et al. [2] present a survey titled "Siren's Song in the AI Ocean" in Computational Linguistics (2025). It provides a taxonomy of fact-conflicting hallucination types and a comprehensive categorisation of the phenomenon. The survey reports that advanced models achieve around 87% on the HaluEval benchmark, showing that detection is

feasible but far from solved. The authors note that mitigating hallucination remains an open research challenge.

Huang et al. [3] provide "A Survey on Hallucination in LLMs: Principles, Taxonomy, Challenges and Open Questions" in ACM Computing Surveys (2025). Their taxonomy spans the training and inference stages of the LLM lifecycle and aggregates detection metrics to guide the design of future retrieval-augmented generation (RAG) systems. A key observation relevant to this project is that the inherent unreliability of LLMs acts as a barrier to industrial innovation.

Qi, Gui, He and Yuan [8] survey automatic hallucination evaluation for natural language generation in IEEE Transactions on Neural Networks and Learning Systems (2024). They provide deep coverage of inconsistencies in generated outputs and of how they can be evaluated automatically, although the survey places less emphasis on concrete mitigation frameworks.

3.3 White-Box Detection from Model Internals [4], [7]

Luan et al. [4] propose "Detecting Hallucination Through Deep Internal Representation Analysis" (IJCAI 2025). Rather than examining generated text, they extract internal representations and apply linear probing, with a Multi-Layer Perceptron used for highly precise detection. The principle is that hidden states encode whether the model "knows" an answer, so a small classifier trained on these states can separate truthful from hallucinated generations. The limitation is that the approach requires full access to the model's internal hidden states, which excludes closed, API-only models.

Barbero et al. [7] present "Hallucination Detection Using Spectral Features of Attention Maps" (EMNLP 2025). They treat attention matrices as graphs and analyse their spectral features, offering a universal, white-box detector that does not need domain adaptation. Its limitation is that it is strictly restricted to transformer-based architectures.

3.4 Efficient Detection Methods [9], [10]

Niu, Haddadi and Pang [9] introduce "Robust Hallucination Detection via Adaptive Token Selection" (ICLR 2025). Instead of processing every token equally, the method adaptively selects the tokens that are informative for detection, which significantly reduces computational cost during inference. A limitation is that accuracy suffers on extremely long text contexts.

Yamada and Arase [10] propose "Light-Weight Detection Using Contrastive Learning for Conditional Text Generation" (ACL 2025). Contrastive learning over conditional generations yields high speed and efficiency. Its drawback is a heavy dependence on specialised negative training pairs, which must be constructed or collected for each task.

These two works are the closest in spirit to the aim of this project because they explicitly target efficiency. They show that detection does not have to rely on repeated sampling, and they supply building blocks reused in the proposed framework.

3.5 Evaluation Metrics and Domain-Specific Work [5], [6]

Boulgouris et al. [6] review faithfulness metrics for hallucination assessment in IEEE Journal of Selected Topics in Signal Processing (2025). The review systematically categorises evaluation tools and benchmarks but concludes that no unified metric applies across all domains.

Wang et al. [5] address hallucination in generated code in "Advancing LLM-Generated Code Reliability: A Hybrid Approach" (IEEE Transactions on Software

Engineering, 2025). The approach combines static analysis and dynamic code execution to identify structural and logical coding hallucinations. Its architectural complexity limits real-time applicability.

3.6 Comparative Analysis

Table 3.1 consolidates the ten studies by method, advantages and limitations.

Sl. No	Paper / Reference	Venue & Year	Method / Technology	Advantages	Limitations
1	Detecting hallucinations in language models using semantic entropy [1]	Nature, 2024	Semantic entropy (bidirectional entailment checks)	High-confidence detection (AUROC 0.790) without task-specific training	High computational overhead; fails on confidently wrong data
2	Siren's Song in the AI Ocean: A Survey on Hallucination [2]	Computational Linguistics, 2025	Taxonomy review of fact-conflicting types	Comprehensive categorisation; advanced models reach ~87% on HaluEval	Mitigation remains an open research challenge
3	A Survey on Hallucination in LLMs: Principles, Taxonomy, Challenges [3]	ACM Computing Surveys, 2025	Comprehensive taxonomy across training and inference	Aggregates detection metrics to guide future RAG designs	Identifies inherent unreliability as a barrier to industrial innovation
4	Detecting Hallucination Through Deep Internal Representation Analysis [4]	IJCAI, 2025	Internal representation extraction using linear probing	Highly precise detection via Multi-Layer Perceptron	Requires full access to the model's internal hidden states
5	Advancing LLM-Generated Code Reliability: A Hybrid Approach [5]	IEEE Trans. on Software Eng., 2025	Static and dynamic hybrid code-execution analysis	Effectively identifies structural and logical coding hallucinations	High architectural complexity for real-time applications
6	A Review of Faithfulness Metrics for Hallucination Assessment [6]	IEEE J. Sel. Top. Signal Process., 2025	Systematic review of semantic faithfulness metrics	Clearly categorises evaluation tools and benchmarks	Lacks a unified metric applicable across all domains
7	Hallucination Detection Using Spectral Features of Attention Maps [7]	EMNLP, 2025	Spectral feature analysis of attention-matrix graphs	Universal, white-box detection without domain adaptation	Strictly limited to transformer-based architectures
8	A Survey of Automatic Hallucination Evaluation [8]	IEEE TNNLS, 2024	Automated evaluation of natural language generation	Deep coverage of inconsistencies in generated outputs	Less emphasis on concrete mitigation frameworks
9	Robust Hallucination Detection via Adaptive Token Selection [9]	ICLR, 2025	Adaptive token-level processing and selection	Significantly reduces computational cost during inference	Struggles to maintain accuracy on extremely long text contexts
10	Light-Weight Detection Using Contrastive Learning [10]	ACL, 2025	Contrastive learning over conditional text generation	High speed and efficiency suitable for conditional generation	Heavily dependent on specialised negative training pairs

Table 3.1: Comparative analysis of hallucination detection literature

3.7 Observations from the Survey

- **Sampling-based methods are accurate but costly** [1], whereas **white-box methods** [4], [7] work in a single pass but need access to model internals.

- **Efficiency research is emerging** [9], [10], yet each approach has a clear weakness (long contexts, specialised negative pairs), and **evaluation is fragmented** across domains [6].
- **Confident errors are under-addressed:** none of the surveyed efficient methods gives an explicit guarantee for confidently wrong responses.

RESEARCH GAP IDENTIFICATION

On reviewing the literature, a research gap concerning efficiency and reliability was identified. Two related limitations emerge, both centred on the foundational method of semantic entropy [1].

4.1 Gap 1: Computational Inefficiency

To calculate semantic entropy, one has to sample responses, typically ten or more per prompt, and then perform additional Natural Language Inference classification or independent verification to group them into meaning clusters. This multi-pass approach is prohibitively expensive for most applications, especially for time-critical systems such as customer-service chatbots, where latency and cost per query are tightly constrained.

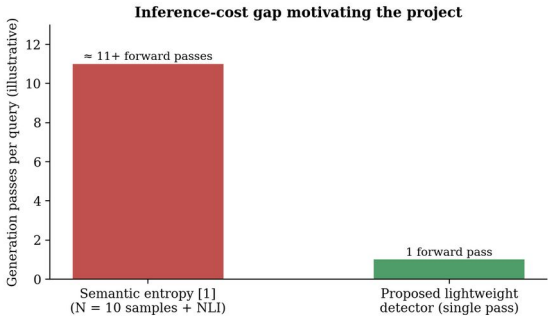


Figure 4.1: Illustrative inference-cost gap between multi-pass and single-pass detection

4.2 Gap 2: Confidently Wrong Responses

Semantic entropy measures consistency. If a model has confidently learned false information and repeatedly outputs the same incorrect answer, all sampled responses fall into a single cluster, semantic entropy is low, and the detector wrongly concludes that the model is confident and factual. In other words, the method provides no guarantee in cases where the error is systematic rather than random. Detecting such cases requires evidence that goes beyond answer consistency, for example signals from internal representations or from a contrastive notion of what a truthful state looks like.

4.3 Summary of the Gap and Research Question

Aspect	Existing approach (semantic entropy)	Desired capability
Inference passes	Multiple samples (10+) per prompt	Single forward pass
Latency	High; unsuitable for time-critical use	Low; suitable for real-time use
Confidently wrong answers	Not detected (low entropy)	Explicitly addressed with a dedicated guard
Guarantees	None for systematic errors	Stronger theoretical guarantees

The research question that this project follows is therefore: **Is it possible to design a lightweight hallucination detection architecture, usable in real-time applications, that does not sample multiple model responses and still offers strong theoretical guarantees in detecting confidently wrong responses?**

METHODOLOGY

The methodology has two parts. The first formalises semantic entropy, which serves as the baseline and conceptual foundation. The second describes the proposed lightweight single-pass framework designed to close the gaps identified in Chapter 4. The overall process follows a structured pipeline of data collection, feature extraction, model design, training, evaluation and deployment.

5.1 Formalising Semantic Entropy

Given a prompt x, an LLM defines a distribution over answer sequences s. Standard predictive entropy is

$$H(S \mid x) = -\Sigma \; p(s \mid x) \cdot \log p(s \mid x)$$

where the sum runs over all possible answer strings. Because many different strings express the same meaning, this quantity mixes uncertainty about wording with uncertainty about facts. Semantic entropy first partitions sampled answers into meaning clusters c using bidirectional entailment, and then computes

$$SE(x) = -\Sigma_c \; p(c \mid x) \cdot \log p(c \mid x), \qquad p(c \mid x) = \Sigma_{s \in c} \, p(s \mid x)$$

Low SE(x) means that probability mass is concentrated in one meaning (a confident, consistent model); high SE(x) means the mass is spread over several incompatible meanings, a signature of hallucination.

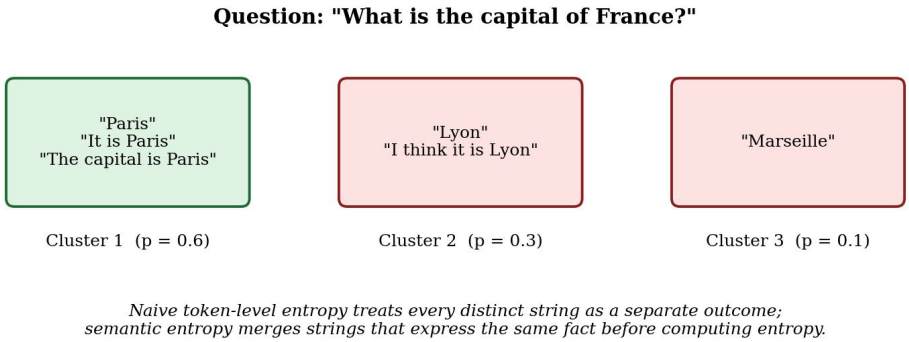


Figure 5.1: Clustering sampled answers by meaning before computing entropy

5.2 Baseline Pipeline

1. **Prompt selection:** take a question from a benchmark with reference answers.
2. **Sampling:** generate N answers (N ≥ 10) at non-zero temperature, recording sequence probabilities.
3. **Entailment checks:** for every pair of answers, test whether each entails the other using an NLI model.
4. **Clustering:** merge answers that entail each other into meaning clusters.
5. **Entropy computation:** estimate cluster probabilities and compute SE(x).
6. **Decision:** flag the answer as likely hallucinated when SE(x) exceeds a threshold chosen on validation data.

5.3 Proposed Lightweight Detection Framework

The proposed framework replaces repeated sampling by a single generation pass during which several cheap signals are collected, and a small fusion head converts them

into a hallucination-risk score. The design draws on complementary strengths identified in the survey.

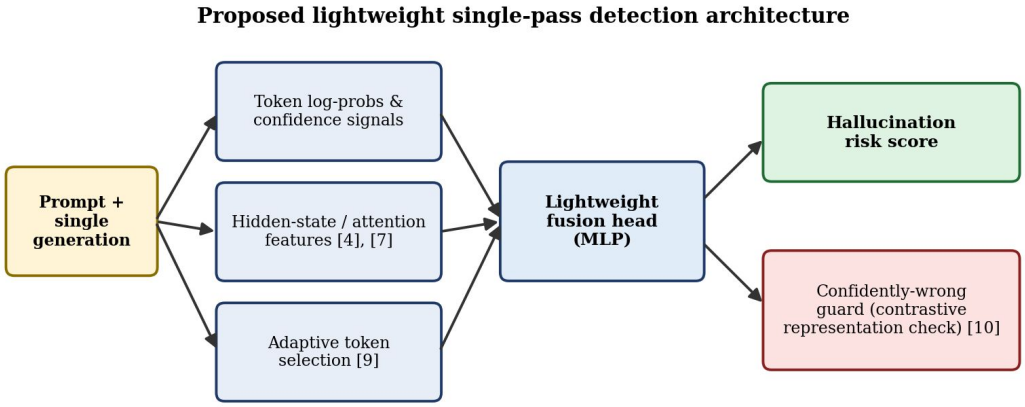


Figure 5.2: Proposed lightweight single-pass hallucination detection architecture

5.3.1 Signal Extraction

- **Token confidence signals:** log-probabilities and local entropy of generated tokens, available at no extra cost during decoding.
- **Internal-representation features:** hidden states from selected layers, following the probing idea of [4], and optionally spectral features of attention maps [7] when model internals are accessible.
- **Adaptive token selection:** only the most informative tokens (for example entity and numeric tokens that carry factual content) are passed to the detector, reducing computation as in [9].

5.3.2 Fusion Head

The selected features are concatenated and passed through a small Multi-Layer Perceptron that outputs a hallucination-risk score between 0 and 1. The head contains only a few thousand to a few hundred thousand parameters, so its cost is negligible compared with generating additional samples.

5.3.3 Confidently-Wrong Guard

To address Gap 2, a contrastive objective in the spirit of [10] is used to train the representation space: hidden states of truthful generations are pulled together, while hidden states of hallucinated generations (including confidently wrong ones) are pushed away. Because this signal is learned from labelled examples of confident errors, it can assign a high risk score even when token probabilities are high and answers are consistent. A known dependency, noted in [10], is the need for suitable negative pairs; these can be built by pairing questions with plausible but incorrect answers.

5.3.4 Training Objective

The fusion head is trained with binary cross-entropy against hallucination labels, combined with a contrastive loss term weighted by a hyperparameter λ:

$$L = L_{BCE} + \lambda \cdot L_{contrastive}$$

5.4 Data Collection and Preprocessing

Training and evaluation require question-answer pairs with correctness or hallucination labels. Typical choices are open-domain QA datasets and hallucination benchmarks such as HaluEval. Preprocessing includes normalising answers, labelling

generated responses as correct or hallucinated by comparison with references (or human judgement), splitting data into training, validation and test sets, and extracting the features described above. A deliberately balanced split is used to avoid a detector that simply predicts the majority class.

5.5 Model Training

- **Optimizer:** Adam with a small learning rate for the fusion head; the base LLM is kept frozen.
- **Regularisation:** dropout and weight decay to prevent overfitting to a single benchmark.
- **Hyperparameters:** layer selection, number of selected tokens, hidden size and λ tuned on the validation set.
- **Early stopping:** based on validation AUROC.

5.6 Evaluation Plan

Metric	Purpose
AUROC	Threshold-independent ability to separate hallucinated from correct answers; the baseline value reported for semantic entropy is 0.790 [1].
Precision, Recall, F1	Quality of detection at the operating threshold, important where false alarms and misses have different costs.
Latency per query	Wall-clock overhead added to generation; the key advantage targeted over multi-pass methods.
Number of forward passes	Direct measure of computational cost (1 for the proposed method versus 10+ for semantic entropy).
Recall on confidently-wrong subset	Performance on cases with low entropy but incorrect answers, the second research gap.

5.7 Challenges and Mitigation

- **Access to model internals:** white-box features need open-weight models; a logit-only variant is kept for restricted settings.
- **Domain shift:** a detector trained on one benchmark may degrade on another, so evaluation across multiple domains is planned.
- **Long contexts:** adaptive token selection may lose accuracy on very long inputs [9]; chunked processing is a possible remedy.
- **Label noise:** automatic labelling of hallucination is imperfect, and the lack of a unified metric [6] complicates comparison.

5.8 Summary of Methodology

In summary, the methodology moves from a precise understanding of semantic entropy to a design that retains the idea of measuring factual rather than linguistic uncertainty, but replaces its expensive multi-sample computation by a single-pass fusion of cheap, complementary signals with an explicit guard for confidently wrong answers.

IMPLEMENTATION

This chapter describes how the baseline and the proposed detector are to be implemented, together with an illustrative worked example. The pseudocode and outputs presented here illustrate the intended behaviour of the method; they are not reported experimental results.

6.1 Implementation Workflow

1. Load an open-weight LLM and a benchmark question set.
2. For each question, generate the model answer and record token log-probabilities and hidden states.
3. For the baseline, additionally sample N answers and run NLI-based clustering.
4. Label each answer as correct or hallucinated.
5. Train the fusion head on the extracted features.
6. Evaluate AUROC, precision, recall, F1 and latency on the held-out test set, comparing with the baseline.

6.2 Baseline: Semantic Entropy Computation

The following Python-style sketch shows how semantic entropy is computed from sampled answers.

```
def semantic_entropy(question, model, nli, n_samples=10):
    answers, probs = model.sample(question, n=n_samples)
    clusters = []                       # each cluster: list of indices
    for i, a in enumerate(answers):
        placed = False
        for c in clusters:
            rep = answers[c[0]]
            if nli.entails(a, rep) and nli.entails(rep, a):
                c.append(i); placed = True; break
        if not placed:
            clusters.append([i])
    mass = [sum(probs[i] for i in c) for c in clusters]
    total = sum(mass)
    p = [m / total for m in mass]
    return -sum(pi * math.log(pi) for pi in p)
```

Note the cost: the model is called N times, and the NLI model is called on answer pairs. This is the overhead that the proposed approach aims to remove.

6.3 Proposed Detector: Single-Pass Scoring

```
class FusionHead(nn.Module):
    def __init__(self, d_hidden, d_tok, d_mid=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_hidden + d_tok, d_mid), nn.ReLU(),
            nn.Dropout(0.2), nn.Linear(d_mid, 1))

    def forward(self, hidden_feats, token_feats):
        x = torch.cat([hidden_feats, token_feats], dim=-1)
        return torch.sigmoid(self.net(x))   # hallucination risk

# inference: one generation pass
```

```
out = llm.generate(prompt, output_hidden_states=True,
                   output_scores=True)
idx = select_informative_tokens(out)          # adaptive selection
risk = head(pool(out.hidden[idx]), tok_stats(out.scores[idx]))
if risk > tau: flag_for_review(out.text)
```

6.4 Illustrative Worked Example

Consider the question "What is the capital of France?" and ten sampled answers. The table shows how semantic entropy treats them; the values are illustrative.

Meaning cluster	Samples	Cluster probability	Interpretation
"Paris" / "It is Paris" / "The capital is Paris"	8	0.80	Dominant, consistent meaning
"Lyon"	1	0.10	Minor competing meaning
"Marseille"	1	0.10	Minor competing meaning

With cluster probabilities 0.8, 0.1 and 0.1, the entropy is $-(0.8 \ln 0.8 + 0.1 \ln 0.1 + 0.1 \ln 0.1) \approx 0.64$ nats. A token-level entropy would be higher because it counts "Paris", "It is Paris" and "The capital is Paris" as distinct outcomes, illustrating why clustering by meaning gives a better estimate of factual uncertainty. In contrast, for an obscure question where ten answers are spread over ten different names, every cluster has probability 0.1 and the entropy is $\ln 10 \approx 2.30$ nats, a clear hallucination signal.

6.5 Expected Behaviour of the Proposed Detector

Scenario	Semantic entropy	Proposed lightweight detector
Model knows the fact	Low entropy: accepted	Low risk score: accepted
Model is guessing (varied answers)	High entropy: flagged	High token-uncertainty and representation signals: flagged
Model is confidently wrong (same wrong answer every time)	Low entropy: missed	Representation / contrastive guard can still raise the risk score: flagged
Time-critical chatbot	10+ passes: too slow	1 pass: real-time suitable

The third row is the central hypothesis of the project. Verifying it experimentally, by measuring recall on a confidently-wrong subset, is the primary evaluation planned for the full implementation.

MAPPING TO SUSTAINABLE DEVELOPMENT GOAL 9

7.1 Overview of SDG 9

United Nations Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) aims to build resilient infrastructure, promote inclusive and sustainable industrialisation, and foster innovation. Modern industries are adopting LLMs as part of their digital infrastructure for data processing, decision making and customer interactions, so the reliability of these models directly affects the resilience of that infrastructure.

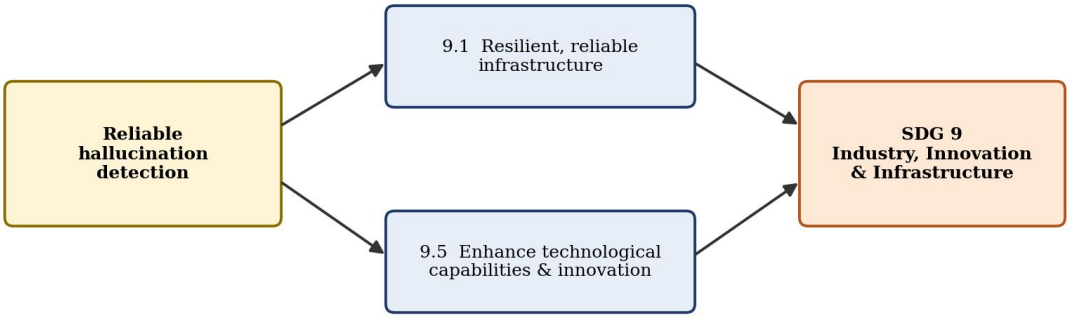


Figure 7.1: Contribution of hallucination detection to SDG 9

7.2 How This Work Contributes

- **Reliable infrastructure:** hallucination is an obstacle to reliable and sustainable industrial innovation. A faulty AI component can cause downstream applications to behave incorrectly and lead to failures in infrastructure [2], [3]. Reliable detection helps contain such failures.
- **Safe innovation:** securing the integrity of AI models enables industries to innovate safely and build automated systems with confidence.
- **Affordable adoption:** because the proposed detector avoids the prohibitive computational costs of multi-pass semantic-entropy approaches, trustworthy AI becomes accessible to organisations that cannot afford order-of-magnitude increases in inference cost.
- **Efficient resource use:** fewer forward passes per query means lower energy and hardware demand at scale, supporting sustainable industrialisation.

7.3 Summary

By addressing the hallucination detection research gap, this project helps provide the reliability necessary to secure sustainable industrial innovation, in line with the objectives of SDG 9.

CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project surveyed the state of hallucination detection in Large Language Models, taking the Nature 2024 semantic entropy paper as its foundation and comparing it with nine further studies spanning taxonomies, white-box probing, spectral attention analysis, adaptive token selection, contrastive learning, faithfulness metrics and code-reliability analysis. Semantic entropy was shown to be a principled and training-free way of separating linguistic uncertainty from factual uncertainty, reaching an AUROC of 0.790.

The analysis also revealed two limitations: the computational burden of sampling ten or more answers and running entailment checks, and the inability to catch confidently wrong responses that are consistently repeated. These limitations define a clear research gap, which was formulated as a research question and answered with a proposed single-pass, lightweight architecture that combines token-confidence signals, internal-representation features, adaptive token selection and a contrastive guard against confident errors.

8.2 Limitations of the Present Work

- The report is based on a literature survey; the proposed architecture is a design whose performance claims still need empirical validation.
- White-box features require open-weight models; fully closed models would rely on weaker, logit-only signals.

8.3 Future Work

1. Implement the proposed detector and benchmark it against semantic entropy on accuracy (AUROC), latency and number of forward passes.
2. Construct and release a confidently-wrong evaluation subset to measure the second research gap directly.
3. Derive formal conditions under which the contrastive guard can bound the miss rate on confident errors, supporting the goal of strong theoretical guarantees.
4. Extend the approach to long contexts and to code generation, building on [5] and [9].
5. Combine detection with mitigation, e.g., trigger retrieval only when the risk score is high.

REFERENCES

[1] J. Farquhar, J. Kenton, A. Mikulik, S. Marcus, and S. Legg, "Detecting hallucinations in language models using semantic entropy," Nature, 2024.

[2] X. Li et al., "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," Computational Linguistics, 2025.

[3] Y. Huang et al., "A Survey on Hallucination in LLMs: Principles, Taxonomy, Challenges and Open Questions," ACM Computing Surveys, 2025.

[4] Z. Luan et al., "Detecting Hallucination Through Deep Internal Representation Analysis," IJCAI, 2025.

[5] X. Wang et al., "Advancing LLM-Generated Code Reliability: A Hybrid Approach," IEEE Transactions on Software Engineering, 2025.

[6] A. Boulgouris et al., "A Review of Faithfulness Metrics for Hallucination Assessment in Large Language Models," IEEE Journal of Selected Topics in Signal Processing, 2025.

[7] F. Barbero et al., "Hallucination Detection Using Spectral Features of Attention Maps," EMNLP, 2025.

[8] S. Qi, L. Gui, Y. He, and Z. Yuan, "A Survey of Automatic Hallucination Evaluation on Natural Language Generation," IEEE Transactions on Neural Networks and Learning Systems, 2024.

[9] M. Niu, H. Haddadi, and G. Pang, "Robust Hallucination Detection via Adaptive Token Selection," ICLR, 2025.

[10] M. Yamada and Y. Arase, "Light-Weight Detection Using Contrastive Learning for Conditional Text Generation," ACL, 2025.